Kurs: Linux + GitHub Actions, Teil 2 (v5)

Teil 2 baut auf Teil 1 auf. Du brauchst die Repository-Struktur aus Teil 1, insbesondere `daten/server.log`. Es gibt **4 Lektionen** und ein Abschlussprojekt.

In GitHub Actions erscheinen Befehlsausgaben aus `run:`-Schritten in den Job-Protokollen; die Laufübersicht zeigt Zusammenfassungen und den Bereich **Artifacts** für hochgeladene Dateien. Für `workflow_dispatch` muss die Workflow-Datei im Standardbranch liegen, damit GitHub **Run workflow** anbietet. In der Actions-Oberfläche wählst du den Branch für den Lauf aus. Dafür brauchst du Schreibzugriff auf dein Lern-Repository.[1]

Vorbereitung

Arbeite in deinem eigenen Lern-Repository aus Teil 1 weiter. Das Kurs-Repository ist die Vorlage; dein Lern-Repository ist der Ausführungsort für die Workflows. `daten/server.log` stammt aus Teil 1 und wird hier weiterverwendet; lege die Datei nicht erneut an.

Die vollständigen Workflow-Dateien stehen im Lerntext und zusätzlich im Kurs-Repository. Lege sie unter dem angegebenen Pfad auf dem Standardbranch deines Lern-Repositorys an und committe sie dort, damit du sie über **Actions** manuell starten kannst. Lizenzhinweise in den Codeblöcken verweisen auf Dateien im Kurs-Repository. Bearbeitest du Dateien direkt im Browser, speichert **Commit changes** auf dem ausgewählten Branch; arbeitest du lokal, pushe deine Commits wie beschrieben.

Lege die beiden folgenden Beispieldateien in deinem Lern-Repository an. Die Ordner entstehen automatisch über den Pfad im Dateinamen.

Datei anlegen: `daten/zugriffe.log` (zehn erfundene Zugriffsdatensätze; die Kommentarzeilen gehören ebenfalls zur Datei):

```
# Linux CI — Beispieldaten
# Lizenz: CC BY 4.0 (siehe LICENSE-CONTENT.txt im Kurs-Repository)
anna /start
ben /produkte
anna /produkte
clara /start
```

[1] GitHub dokumentiert Voraussetzungen für `workflow_dispatch`, Branch-Auswahl und Berechtigungen für manuelle Starts. Quellen: https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow[2] und https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#workflow_dispatch[3] (geprüft am 06.11.2026).

```
anna /kontakt
ben /start
clara /produkte
anna /produkte
ben /kontakt
clara /kontakt
```

Erwartete Seitenaufrufe in den zehn Datensätzen: `/produkte` 4, `/start` 3, `/kontakt` 3. Damit ist die meistbesuchte Seite eindeutig.

Datei anlegen: **`daten/bericht.txt`** (Beispielinhalt für die Dateisuche und das Lesen von Dateien):

```
# Linux CI — Beispieldaten
# Lizenz: CC BY 4.0 (siehe LICENSE-CONTENT.txt im Kurs-Repository)
Quartalsbericht
Umsatz steigt
Kosten sinken
Ausblick positiv
```

In Workflow 06 listet „Dateien finden“ beide Dateien unter `daten/`; „Bericht lesen“ gibt den Inhalt von `daten/bericht.txt` aus.

> In den Logs und Berichten sind keine echten Nutzerdaten oder Zugangsdaten enthalten; verwende für Kursübungen ausschließlich Beispieldaten.

Lektion 5: Mehrere Jobs und `needs:`

Lernziel: Du kannst parallele und voneinander abhängige (`needs:`) Jobs unterscheiden und Werte über `GITHUB_OUTPUT` als Job-Ausgaben weitergeben.

Datei anlegen: `.github/workflows/05-mehrere-jobs.yml`

```yaml
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 05 - Mehrere Jobs

on:
  workflow_dispatch:

jobs:
  zaehlen:
    runs-on: ubuntu-latest
    outputs:
      fehler: ${{ steps.zaehle.outputs.anzahl }}
    steps:
      - uses: actions/checkout@v7

      - name: Fehler zählen
        id: zaehle
        run: |
          ANZAHL=$(grep -c ' ERROR ' daten/server.log || true)
          echo "Gefunden: $ANZAHL"
          echo "anzahl=$ANZAHL" >> "$GITHUB_OUTPUT"

  parallel-a:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Ich laufe gleichzeitig mit zaehlen"

  melden:
    runs-on: ubuntu-latest
    needs: zaehlen
    steps:
      - name: Ergebnis ausgeben
        env:
          FEHLER: ${{ needs.zaehlen.outputs.fehler }}
        run: echo "Der Job zaehlen meldet $FEHLER Fehler"
```

Ausprobieren und kontrollieren: Stelle sicher, dass Workflow 05 und `daten/server.log` im Lern-Repository vorhanden und committed sind. Starte den Workflow unter **Actions**. Im

Graphen können `zaehlen` und `parallel-a` parallel starten; `melden` wartet auf `zaehlen`. Prüfe im Protokoll: `Gefunden: 3` und `Der Job zaehlen meldet 3 Fehler`. `grep` sucht nach `ERROR`, also nach dem Schweregrad zwischen Leerzeichen; Kommentarzeilen werden so nicht mitgezählt.

`actions/checkout@v7` lädt auf dem Runner `ubuntu-latest` die Dateien ins Arbeitsverzeichnis.[4]

Wie das funktioniert

Konzept	Bedeutung
Jobs ohne `needs:`	Starten parallel
`needs: zaehlen`	Wartet auf `zaehlen`. Standardmäßig läuft `melden` nur weiter, wenn `zaehlen` erfolgreich war. Schlägt `zaehlen` fehl, wird `melden` **übersprungen**. Mit besonderen Bedingungen lässt sich das ändern.
Eigene Umgebung	Jeder Job hat einen eigenen Runner. Dateien aus `zaehlen` sind in `melden` **nicht** da.
`echo "x=y" >> "$GITHUB_OUTPUT"`	Schreibt einen Wert für spätere Schritte oder Jobs
`outputs:` am Job	Reicht den Wert an andere Jobs weiter
`needs.zaehlen.outputs.fehler`	So liest der abhängige Job den Wert

Im Tab **Actions** zeigt die Graph-Ansicht des Laufs, wie die Jobs zusammenhängen.

> **`|| true` hier bewusst:** Ohne Treffer gibt `grep -c 0` aus, liefert aber Exit-Code 1 und würde den Schritt fehlschlagen lassen. Hier ist „keine Fehler“ ein gültiges Ergebnis, daher fangen wir den Status ab. Das verschluckt allerdings auch echte `grep`-Fehler (Exit-Code 2), etwa bei einem falschen Dateipfad. Prüfe deshalb sowohl die Fehlermeldung als auch den ausgegebenen Wert im Protokoll.

Aufgaben

1. Stelle sicher, dass Workflow 05 und `daten/server.log` in deinem Lern-Repository vorhanden sind. Starte den Workflow über **Actions**. Sieh dir die Graph-Ansicht an: Welche Jobs können parallel starten?

[4] `actions/checkout@v7` verwendet Node.js 24; die aktuelle Patch-Version ist v7.0.1. Das Major-Tag folgt den Patch-Versionen und pinnt keine davon exakt. Selbst gehostete Runner benötigen mindestens Version 2.327.1. Quellen: https://github.com/actions/checkout/releases[5] und https://github.com/actions/checkout/blob/v7/README.md[6] (geprüft am 06.11.2026).

2. Füge in Workflow 05 deines Lern-Repositorys einen vierten Job `abschluss` hinzu, der erst nach `melden` und `parallel-a` startet: `needs: [melden, parallel-a]`. Committe die Workflow-Änderung auf dem Lern-Branch und starte den Lauf erneut.
3. Ändere in Workflow 05 in deinem Lern-Repository im Job `zaehlen` den Suchbegriff auf `FATAL`. Committe die Änderung im Lern-Repository. Starte den Workflow erneut: Läuft `melden` trotzdem? Was meldet er?
4. Entferne in Workflow 05 den Checkout-Schritt im Job `zaehlen`. Committe und pushe die Änderung auf deinen Lern-Branch, dann starte den Workflow erneut. Im Protokoll erscheint eine `grep`-Fehlermeldung auf stderr. Warum endet der Schritt trotzdem erfolgreich? Prüfe `ANZAHL` und die Ausgabe von `melden`; beide enthalten keinen Zählerwert, denn `ANZAHL` ist leer. Erkläre, wie `|| true` dazu führt, dass der Fehlerstatus abgefangen wird.

Lektion 6: Häufigkeiten zählen und Dateien durchsuchen

Lernziel: Du kannst mit `sort` und `uniq` Häufigkeiten zählen, mit `awk` eine Spalte auslesen und mit `find` Dateien suchen.

Datei anlegen: `.github/workflows/06-auswerten.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 06 - Auswerten

on:
  workflow_dispatch:

jobs:
  auswerten:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Wer war wie oft da?
        run: |
          echo "--- Besuche pro Person ---"
          grep -v '^#' daten/zugriffe.log | cut -d' ' -f1 | sort | uniq -c | sort -nr
```

```
      - name: Welche Seiten sind beliebt?
        run: |
          echo "--- Seitenaufrufe ---"
          grep -v '^#' daten/zugriffe.log | cut -d' ' -f2 | sort | uniq -c | sort -nr

      - name: Top 1
        run: |
          echo "--- Beliebteste Seite ---"
          grep -v '^#' daten/zugriffe.log | cut -d' ' -f2 | sort | uniq -c | sort -
nr | head -n 1

      - name: Mit awk eine Spalte holen
        run: grep -v '^#' daten/zugriffe.log | awk '{print $2}' | sort -u

      - name: Dateien finden
        run: |
          echo "--- alle .log-Dateien ---"
          find . -name "*.log"
          echo "--- alle Dateien in daten/ ---"
          find daten -type f

      - name: Bericht lesen
        run: cat daten/bericht.txt
```

Runner: Der Workflow läuft auf dem GitHub-gehosteten Runner `ubuntu-latest` und verwendet `actions/checkout@v7`.[7]

Ausprobieren und kontrollieren: Stelle sicher, dass Workflow 06, `daten/zugriffe.log` und `daten/bericht.txt` in deinem Lern-Repository vorhanden sind und committe sie auf dem Standardbranch. Starte den Workflow über **Actions → 06 - Auswerten → Run workflow**. Falls die Schaltfläche fehlt, prüfe, ob die Workflow-Datei im Standardbranch liegt. Erwartete Zählungen: `anna` vier Zugriffe, `ben` drei, `clara` drei; `/produkte` vier, `/start` drei und `/kontakt` drei. Die Top-1-Ausgabe ist `/produkte`. `find daten -type f` listet unter anderem `einkauf.txt`,

[7] `actions/checkout@v7` verwendet Node.js 24; die aktuelle Patch-Version ist v7.0.1. Das Major-Tag folgt den Patch-Versionen und pinnt keine davon exakt. Selbst gehostete Runner benötigen mindestens Version 2.327.1. Quellen: https://github.com/actions/checkout/releases[8] und https://github.com/actions/checkout/blob/v7/README.md[9] (geprüft am 06.11.2026).

`server.log`, `zugriffe.log` und `bericht.txt` auf. Anschließend zeigt der Job-Protokollschritt den Inhalt von `daten/bericht.txt`.

Das Zählmuster (merken!)

```
grep -v '^#' datei | cut -d' ' -f1 | sort | uniq -c | sort -nr
   ↓ Kommentare weg  ↓ Spalte holen  ↓   ↓ zählt       ↓ größte zuerst
```

`grep -v '^#'` blendet Kommentarzeilen aus. `cut` teilt die Zugriffszeilen anhand von Leerzeichen in Felder: `-f1` wählt die Person, `-f2` den Seitenpfad. `uniq` erkennt nur **aufeinanderfolgende** gleiche Werte; deshalb sortiert `sort` sie vorher. Probiere es aus: Entferne das `sort`, das unmittelbar vor `uniq -c` steht, und vergleiche die Ausgabe.

> **Gleichstand beachten:** `head -n 1` zeigt immer nur eine Zeile. Hätten zwei Seiten gleich viele Zugriffe, würde der Gleichstand verschwiegen. Schau bei eigenen Daten zuerst die volle Liste an.

Kommandos erklärt

Kommando	Zweck
`uniq -c`	Aufeinanderfolgende gleiche Zeilen zählen
`sort -n`	Numerisch sortieren (sonst steht 10 vor 2)
`sort -r`	Umgekehrte Reihenfolge
`sort -nr`	Numerisch, größte zuerst
`sort -u`	Sortieren und doppelte Zeilen entfernen
`awk '{print $2}'`	Zweite Spalte ausgeben (Trenner: Leerraum, also Leerzeichen oder Tabs)
`find . -name "*.log"`	Dateien mit der Endung `.log` suchen
`find daten -type f`	Alle Dateien im Ordner `daten/` finden (`-type d` beschränkt die Suche auf Ordner)

Kommando	Zweck
`wc -l datei`	Physische Zeilen zählen; auch Kommentarzeilen werden mitgezählt
`grep -v '^#' datei`	Kommentarzeilen aus der Ausgabe herausfiltern

> `find` mit Platzhaltern: Setze das Muster in Anführungszeichen (`"*.log"`), damit die Shell es nicht schon vorher auflöst, sondern `find` es bekommt.

Aufgaben

1. Passe das gezeigte Zählmuster an, nutze `head -n 1` und führe es im Lern-Repository aus. Welche Person hat die meisten Zugriffe? Vergleiche mit der Kontrollzählung.
2. Zähle die Log-Level in der aus Teil 1 kopierten Datei `daten/server.log`. Filtere zuerst die Kommentarzeilen mit `grep -v '^#'` heraus, danach liest `awk '{print $3}'` die dritte Spalte des Logs. Erwartet werden `INFO=5`, `ERROR=3`, `WARNING=1`.
3. Suche im Lern-Repository mit `find daten -type f -name "*.txt"` nach Textdateien im Ordner `daten/`. Erwartet werden `daten/einkauf.txt` und `daten/bericht.txt`.
4. Entferne in deinem Lern-Repository das `sort`, das direkt vor `uniq -c` steht. Beschreibe, was sich ändert: `uniq` zählt dann nur direkt benachbarte gleiche Zeilen.

Lektion 7: Artefakte hochladen und herunterladen

Lernziel: Du kannst einen Bericht als Artefakt hochladen (`upload-artifact`), ihn über die Weboberfläche herunterladen und in einem anderen Job weiterverwenden. Außerdem kannst du Ausgaben mit `>` in eine Datei umleiten oder mit `tee` gleichzeitig in eine Datei und ins Jobprotokoll schreiben.

Datei anlegen: `.github/workflows/07-artefakte.yml`

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 07 - Artefakte
```

```
on:
  workflow_dispatch:

jobs:
  bericht:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Bericht erzeugen
        run: |
          mkdir -p ergebnis
          {
            echo "Auswertung vom $(date +%Y-%m-%d)"
            echo "Fehler im Server-Log: $(grep -c ' ERROR ' daten/server.log || true)"
            echo "Datensätze im Zugriffslog: $(grep -vc '^#' daten/zugriffe.log)"
            echo "Beliebteste Seite:"
            grep -v '^#' daten/zugriffe.log | cut -d' ' -f2 | sort | uniq -c | sort -
nr | head -n 1
          } | tee ergebnis/bericht.txt

      - name: Ergebnis hochladen
        uses: actions/upload-artifact@v7
        with:
          name: auswertung
          path: ergebnis/
          if-no-files-found: error

  weiterverarbeiten:
    runs-on: ubuntu-latest
    needs: bericht
    steps:
      - name: Artefakt für den nächsten Job herunterladen
        uses: actions/download-artifact@v8
        with:
          name: auswertung
          path: weitergabe/
```

```
    - name: Bericht im nächsten Job prüfen
      run: cat weitergabe/bericht.txt
```

Runner und Actions: Der Workflow läuft auf dem von GitHub gehosteten Runner `ubuntu-latest` und verwendet `actions/checkout@v7`, `actions/upload-artifact@v7` sowie `actions/download-artifact@v8`.[10]

Zwei Wege, dasselbe Artefakt zu nutzen:

1. **Selbst herunterladen:** Öffne nach dem Lauf in deinem Lern-Repository die Laufübersicht und scrolle zum Bereich **Artifacts**. Lade `auswertung` als ZIP-Datei herunter, entpacke es auf deinem Rechner und lies `bericht.txt`.
2. **Automatisiert weitergeben:** Der Job `weiterverarbeiten` lädt das Artefakt mit `actions/download-artifact@v8` herunter. Der Job braucht dafür keinen Checkout-Schritt, weil das Artefakt über GitHub zwischen den Jobs übertragen wird. Jeder Job hat eine eigene Arbeitsumgebung.

Erwarteter Berichtinhalt in `bericht.txt` (das Datum ändert sich):

```
Auswertung vom YYYY-MM-DD
Fehler im Server-Log: 3
Datensätze im Zugriffslog: 10
Beliebteste Seite:
      4 /produkte
```

Zwei Kommentarzeilen am Anfang der Datei `daten/zugriffe.log` kennzeichnen Herkunft und Lizenz; sie sind keine Zugriffsdatensätze. Der Bericht zählt daher nur Zeilen, die nicht mit `#` beginnen. `grep -vc '^#'` zählt diese Zugriffsdatensätze; `wc -l` würde auch die Kommentarzeilen mitzählen.

> Nach Ablauf der Aufbewahrungsfrist werden Artefakte gelöscht. `retention-days` kann bei `upload-artifact@v7` 1 bis 90 Tage betragen. Richtlinien für Repository, Organisation oder Enterprise können die Höchstfrist verkürzen. Geänderte Werte gelten nur für neu erstellte Artefakte. Artefakte sind nicht für Geheimnisse oder echte Nutzerdaten gedacht.[a]
>
> `grep -c ... || true` erlaubt eine Null bei keinen Treffern, fängt aber auch echte `grep`-Fehler

[10] Die Releases am Prüfdatum sind checkout v7.0.1, upload-artifact v7.0.2 und download-artifact v8.0.2; alle verwenden Node.js 24. Die Major-Tags pinnen keine exakte Patch-Version. Quellen: https://github.com/actions/checkout/releases[11] , https://github.com/actions/upload-artifact/releases[12] , https://github.com/actions/download-artifact/releases[13] und https://github.com/actions/runner/releases[14] (geprüft am 06.11.2026).

> ab. Für eigene Workflows solltest du Fehler getrennt behandeln.
>
> [a] GitHub dokumentiert, dass `retention-days` höchstens 90 Tage betragen darf, Änderungen für neu erstellte Artefakte gelten und Richtlinien für Repository, Organisation oder Enterprise das Maximum begrenzen können. Quellen: https://github.com/actions/upload-artifact/blob/v7/README.md[15] und https://docs.github.com/en/actions/how-tos/writing-workflows/choosing-what-your-workflow-does/storing-and-sharing-data-from-a-workflow[16] (geprüft am 06.11.2026).

So funktioniert es

Element	Bedeutung
`tee datei`	Schreibt die Eingabe in eine Datei und zugleich ins Jobprotokoll
`wc -l < datei`	Zeilen zählen ohne Dateinamen auszugeben
`actions/upload-artifact@v7`	Hängt Dateien als Artefakt an den Workflow-Lauf an
`actions/download-artifact@v8`	Lädt ein Artefakt im Arbeitsverzeichnis eines Jobs herunter[17]
`needs: bericht`	Wartet auf den Job `bericht`

Beim Download des Artefakts aus demselben Workflow-Lauf ist kein zusätzliches Token nötig.[19]

Aufgaben

1. Starte den Workflow in deinem Lern-Repository. Unter **Artifacts** in der Laufübersicht kannst du das Artefakt `auswertung` als ZIP auf deinen Rechner herunterladen. Entpacke es und lies die Datei `bericht.txt`. Der erwartete Inhalt ist oben gezeigt. Falls **Run workflow** nicht angeboten wird, prüfe, ob die Workflow-Datei auf dem Standardbranch liegt.
2. Öffne im Lern-Repository den Job `weiterverarbeiten` und prüfe, dass `bericht.txt` ausgegeben wird. Erkläre, warum der Job `needs: bericht` braucht.

[17] GitHub beschreibt den Austausch von Artefakten zwischen Jobs über Upload- und Download-Actions und erwähnt, dass für Download eines im selben Workflow-Lauf gespeicherten Artefakts kein Token erforderlich ist. Quelle: https://docs.github.com/en/actions/how-tos/writing-workflows/choosing-what-your-workflow-does/storing-and-sharing-data-from-a-workflow[18] (geprüft am 06.11.2026).

[18] https://docs.github.com/en/actions/how-tos/writing-workflows/choosing-what-your-workflow-does/storing-and-sharing-data-from-a-workflow

[19] GitHub beschreibt den Austausch von Artefakten zwischen Jobs über Upload- und Download-Actions und erwähnt, dass für Download eines im selben Workflow-Lauf gespeicherten Artefakts kein Token erforderlich ist. Quelle: https://docs.github.com/en/actions/how-tos/writing-workflows/choosing-what-your-workflow-does/storing-and-sharing-data-from-a-workflow[20] (geprüft am 06.11.2026).

3. Ergänze im Workflow deines Lern-Repositorys eine Zeile mit der Anzahl der `WARNING`-Einträge. Committe und pushe die Änderung auf den Standardbranch, dann starte den Workflow erneut. Kontrolliere, dass die neue Zeile im Browser-Download und im Job `weiterverarbeiten` erscheint.
4. Ergänze im `with:`-Block von `actions/upload-artifact@v7` den Wert `retention-days: 1`. Committe und pushe die Änderung auf den Standardbranch deines Lern-Repositorys und starte den Workflow erneut. Prüfe die Aufbewahrungsfrist, falls GitHub sie in der Laufübersicht anzeigt. Der Wert gilt nur für neu erstellte Artefakte und kann durch Richtlinien für Repository, Organisation oder Enterprise begrenzt sein.[21]
5. Ändere im Workflow `path:` auf den mehrzeiligen Wert und lade zusätzlich `daten/server.log` hoch. Committe die Änderung auf deinem Lern-Branch. Prüfe, dass beide Dateien im ZIP-Download enthalten sind. Lade keine Geheimnisse oder persönlichen Daten hoch.

Mit `>` statt `tee`: `>` schreibt die Ausgabe nur in die Datei und überschreibt einen vorhandenen Inhalt; `tee` schreibt in die Datei und zugleich ins Jobprotokoll.

```
echo "Hallo" > ergebnis/datei.txt     # nur in die Datei, keine Ausgabe im Log
echo "Hallo" | tee ergebnis/datei.txt # in Datei und ins Log
```

Lektion 8: Zeitplan mit Cron und weitere Trigger

Lernziel: Du kannst die Auslöser `workflow_dispatch`, `push`, `pull_request` und `schedule` unterscheiden sowie einfache Cron-Zeitpläne lesen. Workflow 08 demonstriert diese Trigger. Die Workflows 05 bis 07 startest du mit `workflow_dispatch` manuell. Workflow 09 bekommt im Abschlussprojekt zusätzlich einen `push`-Trigger.

Datei anlegen: `.github/workflows/08-zeitplan.yml`

Committe die Datei auf dem Standardbranch, damit `workflow_dispatch` verfügbar ist. Die zusätzlichen Trigger dieser ausdrücklich beauftragten Ausnahme werden unten erläutert.

[21] GitHub dokumentiert, dass `retention-days` höchstens 90 Tage betragen darf, Änderungen für neu erstellte Artefakte gelten und Richtlinien für Repository, Organisation oder Enterprise das Maximum begrenzen können. Quellen: https://github.com/actions/upload-artifact/blob/v7/README.md[22] und https://docs.github.com/en/actions/how-tos/writing-workflows/choosing-what-your-workflow-does/storing-and-sharing-data-from-a-workflow[23] (geprüft am 06.11.2026).

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 08 - Zeitplan

on:
  workflow_dispatch:
  push:
    paths:
      - "daten/**"
  pull_request:
    paths:
      - "daten/**"
  schedule:
    - cron: "30 6 * * 1"

jobs:
  taeglich:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Was hat mich ausgelöst?
        env:
          EREIGNIS: ${{ github.event_name }}
        run: |
          echo "Ausgelöst durch: $EREIGNIS"

      - name: Log-Auswertung
        run: |
          echo "Log-Einträge: $(grep -c -E ' (INFO|WARNING|ERROR) ' daten/server.log)"
          echo "Datum (UTC): $(date -u +%Y-%m-%d)"
```

Ausprobieren: Starte den Workflow über **Actions → 08 - Zeitplan → Run workflow** manuell und prüfe im Job-Protokoll `Ausgelöst durch: workflow_dispatch`. Erstelle für den `push`-Test über die Branch-Auswahl auf dem Tab **Code** einen Arbeitsbranch vom Standardbranch. Ändere dort eine Datei unter `daten/`, committe und pushe die Änderung, um den `push`-Trigger zu testen. Im Job-Protokoll sollte `Ausgelöst durch: push` stehen. Erstelle für den `pull_request`-Trigger einen Pull Request mit Änderungen unter `daten/`. Öffne danach die zugehörige

Laufübersicht und prüfe, ob GitHub einen Lauf gestartet hat.

Trigger im Überblick

Trigger	Wann läuft der Workflow?
`workflow_dispatch`	Über **Run workflow** manuell starten; Workflow-Datei muss auf dem Standardbranch liegen, Branch für den Lauf auswählen
`push`	Startet bei Push-Vorgängen mit Änderungen an Dateien unter `daten/` (`paths:`-Filter); die Workflow-Datei muss auf dem gepushten Branch liegen
`pull_request`	Startet bei Pull Requests mit Änderungen an Dateien unter `daten/` (`paths:`-Filter); benötigt einen Pull Request von einem Arbeitsbranch zum Standardbranch
`schedule`	Zeitgesteuert; Workflow-Datei und Ausführung liegen auf dem Standardbranch

Cron-Syntax: 5 Felder

```
30   6    *    *    1
|    |    |    |    └── Wochentag (0-6, Sonntag = 0, Montag = 1)
|    |    |    └─────── Monat (1-12)
|    |    └──────────── Tag im Monat (1-31)
|    └───────────────── Stunde (0-23)
└────────────────────── Minute (0-59)
```

`30 6 * * 1` bedeutet: Jeden Montag um 06:30 Uhr **UTC**.

Cron	Bedeutung
`0 * * * *`	Jede volle Stunde
`*/15 * * * *`	Alle 15 Minuten
`0 8 * * 1-5`	Werktags um 08:00
`0 0 1 * *`	Am Monatsersten um Mitternacht

Wichtig zu `schedule`:

- Dieser Workflow ist eine ausdrücklich beauftragte Ausnahme von der Kursregel: Er zeigt neben `workflow_dispatch` auch `push`, `pull_request` und `schedule`.
- Ohne Zeitzonenangabe gelten die Zeiten in **UTC**, nicht in deiner lokalen Zeit. GitHub erlaubt auch eine IANA-Zeitzone, zum Beispiel `timezone: "America/New_York"`. In diesem Kurs verwenden wir UTC.[24]
- Geplante Workflows laufen nur, wenn die Workflow-Datei auf dem **Standardbranch** liegt. Sie verwenden den neuesten Commit dieses Branches. Ein manueller Lauf mit `workflow_dispatch` testet daher nicht den Zeitplan selbst.[26]
- Ein Zeitplan startet nicht garantiert genau zum angegebenen Zeitpunkt. Bei hoher Last können Läufe verzögert werden; bei sehr hoher Last können wartende Läufe verworfen werden. Besonders stark belastet ist oft der Beginn einer Stunde.[28]
- In öffentlichen Repositories deaktiviert GitHub geplante Workflows automatisch, wenn das Repository 60 Tage inaktiv war. Du kannst sie anschließend im Actions-Tab wieder aktivieren.[30]
- Der kürzeste Abstand laut GitHub-Dokumentation ist 5 Minuten.[32]

Aufgaben

1. Erstelle über die Branch-Auswahl auf dem Tab **Code** einen Arbeitsbranch vom Standardbranch, der Workflow 08 enthält. Ändere in diesem Branch `daten/einkauf.txt`, committe und pushe die Änderung. Prüfe im Job-Protokoll, dass `Ausgelöst durch: push` erscheint. Alternativ speichert **Commit changes** im Browser direkt auf dem ausgewählten Branch und löst den Push aus.
2. Schreibe Cron-Ausdrücke für:
   - jeden Tag um 12:00 UTC
   - jeden Freitag um 17:45 UTC

[24] Die Workflow-Syntax dokumentiert UTC als Standardzeitzone, optionale IANA-Zeitzonen, Ausführung auf dem Standardbranch und einen Mindestabstand von fünf Minuten. Quelle: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onschedule[25] (geprüft am 06.11.2026).

[26] Die Workflow-Syntax dokumentiert UTC als Standardzeitzone, optionale IANA-Zeitzonen, Ausführung auf dem Standardbranch und einen Mindestabstand von fünf Minuten. Quelle: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onschedule[27] (geprüft am 06.11.2026).

[28] GitHub weist auf mögliche Verzögerungen oder verworfene Läufe bei Last sowie die automatische Deaktivierung geplanter Workflows nach 60 Tagen Inaktivität öffentlicher Repositorys hin. Quelle: https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#schedule[29] (geprüft am 06.11.2026).

[30] GitHub weist auf mögliche Verzögerungen oder verworfene Läufe bei Last sowie die automatische Deaktivierung geplanter Workflows nach 60 Tagen Inaktivität öffentlicher Repositorys hin. Quelle: https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#schedule[31] (geprüft am 06.11.2026).

[32] Die Workflow-Syntax dokumentiert UTC als Standardzeitzone, optionale IANA-Zeitzonen, Ausführung auf dem Standardbranch und einen Mindestabstand von fünf Minuten. Quelle: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax#onschedule[33] (geprüft am 06.11.2026).

- alle 30 Minuten Vergleiche deine Cron-Ausdrücke mit den Beispielen in der Tabelle oben.

3. **Zeitplan praktisch prüfen:** Die Datei `.github/workflows/08-zeitplan.yml` muss auf dem Standardbranch deines Lern-Repositorys liegen; der Schedule-Trigger verwendet den neuesten Commit dieses Branches. Öffne die Datei im Browser, klicke auf das Stiftsymbol und ändere `cron: "30 6 * * 1"` vorübergehend zu `cron: "*/5 * * * *"`. Wähle **Commit changes** auf dem Standardbranch. Warte einige Minuten und prüfe unter **Actions**, ob ein Lauf mit `Ausgelöst durch: schedule` erscheint. Ein manueller `workflow_dispatch`-Start testet den Zeitplan nicht; geplante Läufe können verzögert sein oder ausbleiben. Stelle danach im Browser den ursprünglichen Cron-Ausdruck `30 6 * * 1` wieder her und wähle erneut **Commit changes** auf dem Standardbranch. Für dieses Lern-Repository brauchst du keinen Git Flow.

Abschlussprojekt: Der Auto-Prüfer

Richte dich für die Branch-Erstellung nach der Vorbereitung von Teil 1.

Verbinde die Lektionen zu einem Workflow, der Aufgaben **automatisch prüft**. Der Workflow hat zwei Testwege: manueller Start über `workflow_dispatch` und automatischer Start bei einem `push` mit Änderungen unter `loesungen/`.

Datei anlegen: `.github/workflows/09-pruefer.yml`

Lege den gezeigten Workflow im Lern-Repository an und committe ihn auf den Standardbranch, damit `workflow_dispatch` verfügbar ist. Erstelle anschließend über die Branch-Auswahl auf dem Tab **Code** einen Arbeitsbranch vom Standardbranch. Dieser Branch enthält den Workflow ebenfalls.

```
# Linux CI – Beispielcode
# Lizenz: MIT (siehe LICENSE-CODE.txt im Kurs-Repository)
name: 09 - Auto-Prüfer

on:
  workflow_dispatch:
  push:
    paths:
```

```
      - "loesungen/**"

jobs:
  pruefen:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7

      - name: Aufgabe 1 prüfen
        run: |
          ERWARTET=$(grep -c ' ERROR ' daten/server.log)
          ANTWORT=$(tr -d '[:space:]' < loesungen/aufgabe1.txt)
          echo "Erwartet: $ERWARTET, Antwort: $ANTWORT"
          if [ "$ANTWORT" = "$ERWARTET" ]; then
            echo "Richtig!"
          else
            echo "Leider falsch. Tipp: grep -c"
            exit 1
          fi
```

Lege auf diesem Arbeitsbranch `loesungen/aufgabe1.txt` mit der Beispielantwort `3` an und committe die Datei auf diesem Branch. Wenn du **Add file → Create new file** verwendest, entsteht der Ordner `loesungen/` über den Dateipfad. Pushe den Commit, damit der `push`-Trigger startet.

Der Auto-Prüfer in Workflow 09 ist eine ausdrücklich beauftragte Ausnahme vom sonst manuellen Start: Er startet zusätzlich bei Änderungen unter `loesungen/`.[34] Prüfe im Job-Protokoll des Schritts `Aufgabe 1 prüfen`, dass `Erwartet: 3, Antwort: 3` und `Richtig!` ausgegeben werden.

Der Workflow verwendet den GitHub-gehosteten Runner `ubuntu-latest` und `actions/checkout@v7` (Node.js 24). Das Major-Tag `@v7` folgt stets der neuesten Patch-Version seiner Hauptversion; es fixiert keinen einzelnen Patch. GitHub verwaltet den Runner für `ubuntu-latest`. Selbst gehostete Runner benötigen für Node.js-24-Actions mindestens Runner-Version 2.327.1.[37]

[34] GitHub dokumentiert manuelle Workflow-Starts, die Branch-Auswahl beim Start und den `push`-Trigger. Quellen: https://docs.github.com/en/actions/how-tos/manage-workflow-runs/manually-run-a-workflow[35] und https://docs.github.com/en/actions/reference/workflows-and-actions/events-that-trigger-workflows#push[36] (geprüft am 06.11.2026).

[37] `actions/checkout@v7` verwendet Node.js 24; die aktuelle Patch-Version ist v7.0.1. Das Major-Tag folgt den Patch-Versionen und pinnt keine davon exakt. Selbst gehostete Runner benötigen mindestens Version 2.327.1. Quellen: https://github.com/actions/checkout/releases[38] und https://github.com/actions/checkout/blob/v7/README.md[39] (geprüft am 06.11.2026).

Das Prinzip

- Der Prüfer berechnet die richtige Antwort **selbst** aus den Daten
- Er liest die Antwort aus der Lösungsdatei und vergleicht beides
- `exit 1` lässt den Schritt rot werden, `exit 0` (oder kein `exit`) grün
- `tr -d '[:space:]'` entfernt Leerzeichen, Tabs und Zeilenumbrüche, damit `3` und `3` mit abschließendem Zeilenumbruch gleich behandelt werden

Element	Bedeutung
`[ "$A" = "$B" ]`	Texte vergleichen (Leerzeichen innerhalb der Klammern sind Pflicht)
`if ... then ... else ... fi`	Bedingung
`exit 1`	Mit Fehlerstatus beenden
`tr -d`	Zeichen löschen

> `|| true` fängt auch hier echte `grep`-Fehler ab (siehe Lektion 5). Für den Kurs ist das eine bewusste Vereinfachung.

Aufgaben

1. Ändere `loesungen/aufgabe1.txt` auf `5`, committe und pushe die Änderung auf den Arbeitsbranch, der die Workflow-Datei enthält. Der Prüfschritt sollte rot werden.
2. Korrigiere die Antwort auf `3`, committe und pushe erneut. Der Prüfschritt sollte grün werden.
3. Erweitere im Lern-Repository den Prüfer um Aufgabe 2: „Wie viele Einkaufseinträge enthält `daten/einkauf.txt`?“ Filtere die Kommentarzeilen mit `grep -v '^#'` heraus. Lege `loesungen/aufgabe2.txt` mit der Antwort `5` an, committe und pushe die Änderung.
4. Ergänze im Lern-Repository Aufgabe 3 mit einer eigenen Frage auf Basis von `daten/zugriffe.log`. Filtere Kommentarzeilen heraus, lege `loesungen/aufgabe3.txt` an, committe und pushe die Änderung.

Häufige Fehler in Teil 2

Fehler	Ursache
`needs`-Job startet nicht	Der vorherige Job ist fehlgeschlagen oder wurde übersprungen
Output ist leer	`id:` am Schritt fehlt oder der Name in `outputs:` stimmt nicht. Fehlt die Datei, meldet `grep` den Fehler auf stderr; der Ausdruck `true` nach dem OR-Operator lässt den Schritt trotzdem erfolgreich enden und `ANZAHL` bleibt leer.
Dateien fehlen im nächsten Job	Jeder Job hat eine eigene Arbeitsumgebung. Dateien zwischen Jobs mit Artefakten weitergeben.
`uniq -c` zählt falsch	`sort` davor vergessen
Artefakt nicht zu finden	`path:` zeigt auf keinen vorhandenen Datei- oder Ordnerpfad
Geändertes `retention-days` wirkt nicht	Der neue Wert gilt nur für nach der Änderung erstellte Artefakte; Workflow erneut starten
Häufigkeitswert zu hoch	Beispieldatei enthält Kommentarzeilen; vor der Auswertung mit `grep -v '^#'` herausfiltern
Beispieldatei fehlt im Runner	Datei ist lokal, aber noch nicht in den ausgewählten Branch des Lern-Repositorys gepusht
Zeitplan läuft nicht	Schedule-Datei fehlt auf dem Standardbranch, UTC-Zeit falsch gerechnet, Lauf verzögert oder öffentliches Repo inaktiv
Push-Lauf startet nicht	Änderung liegt außerhalb des `paths:`-Filters oder die Workflow-Datei fehlt auf dem gepushten Branch
Prüfer meldet “falsch” bei richtiger Antwort	Leerzeichen oder Zeilenumbruch in der Antwortdatei

Spickzettel-Ergänzung (`CHEATSHEET.md`)

```
needs: job                       Job wartet auf anderen Job
echo "k=v" >> "$GITHUB_OUTPUT"   Wert weitergeben
sort | uniq -c | sort -nr        Häufigkeiten zählen
awk '{print $2}'                 Spalte 2 ausgeben
find . -name "*.log"             Dateien suchen
cmd | tee datei                  In Datei und Log schreiben
wc -l < datei                    Zeilen ohne Dateinamen
actions/upload-artifact@v7      Artefakt hochladen
actions/download-artifact@v8    Artefakt im Job laden
cron: "min std tag mon wtag"     Zeitplan (ohne Angabe: UTC)
[ "$A" = "$B" ]                  Texte vergleichen
exit 1                           Schritt bewusst fehlschlagen lassen
```